PEGASO
Università Telematica

TEMPLATE
PROJECT WORK

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole *(pari a circa Minimo 12 – Massimo 20 pagine)*
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Alex Di Paolo
Numero di matricola	0312300365
Tema n. (Indicare il numero del tema scelto):	5
Titolo del tema (Indicare il titolo del tema scelto):	Machine Learning per Processi Aziendali
Traccia del PW n. (Indicare il numero della traccia scelta):	18
Titolo della traccia (Indicare il titolo della traccia scelta):	Triage automatico dei ticket con Machine Learning
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	AutoTriage NLP: Sistema intelligente di classificazione e prioritarizzazione ticket per l'assistenza aziendale

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio
(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

Al giorno d'oggi, la velocità di risposta e l'ottimizzazione dei flussi informativi rappresentano requisiti fondamentali per la competitività di qualsiasi realtà aziendale. Il seguente elaborato si propone di studiare e implementare una soluzione pratica basata su tecniche di *Machine Learning* per l'ottimizzazione del supporto clienti.

A fronte di ciò, il progetto **AutoTriage NLP** si propone di automatizzare il processo di smistamento dei ticket, considerando sia le complessità tecniche dell'analisi del linguaggio naturale, sia i vincoli normativi e organizzativi tipici di un'impresa moderna. Per raggiungere lo scopo, è stato richiesto di creare un'architettura software completa, che spazia dall'ingegnerizzazione di dati reali fino alla realizzazione di un'interfaccia utente interattiva.

L’obiettivo dell'elaborato è dimostrare la capacità di generare, manipolare e analizzare flussi di dati testuali non strutturati attraverso il linguaggio di programmazione **Python**. Questo processo di sviluppo non è stato solo un esercizio tecnico, ma il punto di convergenza di un percorso formativo multidisciplinare: lo sviluppo dell’elaborato è stato infatti affrontato mobilitando risorse, strumenti e metodologie apprese durante il Corso di Laurea in **Informatica per le Aziende Digitali (L-31)**, affinché si potesse raggiungere lo scopo finale con rigore scientifico.

Di seguito viene spiegato nel dettaglio come i singoli insegnamenti sono stati utilizzati concretamente per costruire le diverse parti del sistema.

1. Area Programmazione e Algoritmi

Il pilastro tecnico del progetto si fonda sulle competenze acquisite nei corsi di programmazione.

- **Programmazione 2**: Questo insegnamento è stato fondamentale, in quanto l’intero progetto è scritto in Python. Le lezioni sulla gestione delle stringhe, sulle strutture dati (liste, dizionari) e sulle librerie per la *Data Science* sono state applicate direttamente nella scrittura del codice. In particolare, l’uso delle librerie **Pandas** e **NumPy** per la manipolazione del dataset Kaggle (prepare_data.py) e **Matplotlib / Seaborn** per la visualizzazione dei grafici nella dashboard derivano direttamente dai moduli di questo corso. La gestione delle eccezioni (try-except) implementata nella pipeline di traduzione automatica è un’applicazione diretta delle *best practices* apprese.
- **Programmazione 1**: Sebbene focalizzato sul linguaggio C, questo corso ha fornito la *forma mentis* algoritmica necessaria per il *problem solving*. La logica strutturata utilizzata per costruire la pipeline di preprocessing (cicli, controlli condizionali per la pulizia del testo) e la modularizzazione del codice in file separati (src/) riflettono i principi di programmazione strutturata e modulare appresi in questo insegnamento.
- **Algoritmi e Strutture Dati**: La scelta delle strutture dati per gestire i vocabolari e le matrici sparse (dizionari / hash map) stata guidata dalla necessità di ottimizzare i tempi di accesso. Inoltre, la comprensione della complessità computazionale è stata utile per garantire che la funzione di traduzione e batch processing potesse processare file CSV di grandi dimensioni in tempi ragionevoli, parallelizzando le operazioni tramite *ThreadPoolExecutor* per evitare colli di bottiglia nell’I/O.

2. Area Ingegneria del Software e Metodologie

Il codice non è scritto in modo monolitico, ma ingegnerizzato per essere manutenibile.

- **Ingegneria del Software**: Questo corso è stato cruciale per l’impostazione metodologica. I concetti relativi al ciclo di vita del software, alla manutenibilità e alla modularità hanno guidato la divisione del progetto in componenti logiche distinte (*Data Layer, Model Layer, Frontend Layer*). Anche se non esplicitamente disegnati, i principi del design pattern sono stati applicati nell’architettura UnifiedModel, che incapsula la complessità di due classificatori paralleli in un’unica interfaccia.

- **Basi di Dati**: Anche se il progetto utilizza file CSV per la portabilità non un DBMS SQL attivo, le competenze di modellazione dei dati sono state essenziali. La ristrutturazione del dataset *Kaggle* nel file tickets_it_augmented.csv (con colonne *ID, title, body, text, category e priority*) è stata progettata seguendo i principi di normalizzazione e integrità dei dati appresi nel corso, garantendo che il dataset di training fosse coerente, privo di duplicati e semanticamente valido.

3. Area Matematica e Statistica

Il cuore "intelligente" del progetto (il *Machine Learning*) si basa su solide fondamenta matematiche.

- **Calcolo delle Probabilità e Statistica**: Questo insegnamento è la base teorica del *Machine Learning*. Concetti come distribuzione di probabilità, frequenza (alla base del TF-IDF), correlazione e inferenza statistica sono stati indispensabili per comprendere come il modello di Regressione Logistica stima la probabilità di appartenenza a una classe. Inoltre, le metriche di valutazione utilizzate nel report (Accuracy, Precision, Recall, F1-Score) e la lettura della matrice di confusione sono applicazioni dirette degli argomenti trattati in questo corso.
- **Matematica Discreta e Analisi Matematica**: La vettorizzazione del testo crea spazi vettoriali multidimensionali. La comprensione delle operazioni su matrici e vettori (algebra lineare) è stata necessaria per capire come l'algoritmo calcola le distanze e gli iperpiani di separazione tra le categorie. L'introduzione di *LIME* per l'interpretabilità si basa proprio su perturbazioni locali dello spazio vettoriale.

4. Area Web e Tecnologie di Rete

L'interfaccia utente è una *web application* eseguita su browser.

- **Tecnologie Web**: Le competenze su HTML e CSS sono state utilizzate per la personalizzazione dell'interfaccia grafica. Il file style.css (con l'uso di variabili CSS :root, selettori e proprietà avanzate) è frutto delle lezioni su CSS. Inoltre, la comprensione del protocollo HTTP e del modello Client-Server è stata utile per capire come **Streamlit** gestisce le richieste dell'utente e come integrare le API di traduzione esterna (deep-translator) nel flusso dati.

5. Area Giuridica ed Economica

Il progetto non è solo tecnico, ma risolve un problema aziendale nel rispetto delle norme.

- **Diritto per le Aziende Digitali**: Questo corso ha avuto un impatto decisivo sulla scelta della fonte dati. Consapevole delle normative sulla Privacy e sul **GDPR** (Regolamento UE 2016/769) studiate nel corso, e in particolare del principio di "Data Minimization", ho scelto deliberatamente di non raccogliere dati reali da aziende terze, operazione che avrebbe comportato complessi oneri di compliance e rischi di data breach. Al contrario, ho adottato un approccio "Privacy by Design" utilizzando un *Dataset Pubblico Open Source (Kaggle)*, già anonimizzato alla fonte. Questa scelta metodologica garantisce la totale assenza di Dati Personali Identificabili (PII) nel sistema, rendendo il progetto intrinsecamente sicuro e compliant by design.

- **Strategia, Organizzazione e Marketing**: La contestualizzazione del progetto nasce dall'analisi dei processi aziendali. I concetti di efficienza operativa, gestione delle risorse e ottimizzazione dei flussi di lavoro (*workflow*) studiati in questo corso hanno fornito la giustificazione economica per lo sviluppo di un sistema di Triage automatico. Il calcolo del ROI dimostra come l'automazione non sia solo tecnologia, ma leva strategica per **fluidificare** i colli di bottiglia organizzativi.

In sintesi, l'elaborato rappresenta una sintesi pratica in cui l'algoritmica (programmazione) incontra la matematica (statistica / ML), si veste di un'interfaccia fruibile (tecnologie web) e si inserisce in un contesto legale e organizzativo corretto (diritto ed economia).

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato
(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

Il progetto è stato sviluppato in un arco temporale di circa **4 settimane**, per un impegno complessivo di circa **80 ore** di lavoro. L'attività è stata strutturata in cinque fasi operative distinte, ciascuna finalizzata al raggiungimento dei requisiti richiesti dalla traccia prescelta.

Fase 1: Selezione traccia, ricerca bibliografica e setup (10 ore)

- **Attività**: Questa fase iniziale è stata dedicata all'analisi critica delle varie tracce proposte, portando alla scelta definitiva del progetto "AutoTriage NLP", per la sua rilevanza nel contesto aziendale moderno. Si è proceduto con la raccolta e lo studio del materiale didattico, consultando le risorse della biblioteca online della piattaforma **UniPegaso** (in particolare la sezione *Pearson Bookshelf*) e le dispense del corso di *Programmazione 2*. Fondamentale è stata la partecipazione agli incontri di **didattica interattiva** offerti dall'Ateneo, utili per chiarire dubbi procedurali e ricevere feedback preliminari. Parallelamente, è stato configurato l'ambiente di sviluppo sulla piattaforma **Antygravity** per la stesura del codice.
- **Difficoltà**: Orientarsi tra le diverse risorse disponibili e definire il perimetro teorico del progetto per non eccedere nella complessità tecnica rispetto ai tempi previsti.
- **Superamento**: La selezione mirata delle fonti e il confronto con i docenti durante le sessioni interattive hanno permesso di delineare chiaramente gli obiettivi e gli strumenti necessari per focalizzare gli obiettivi sulla realizzazione di un prototipo funzionante.

Fase 2: Analisi del dominio e Data Engineering (20 ore)

- **Attività**: In questa fase è stato condotto uno studio approfondito del problema del *Ticket Triage* e dei flussi di lavoro tipici di un Help Desk aziendale. Per superare i limiti dei generatori casuali e garantire il massimo realismo, si è scelto di utilizzare il dataset reale *Customer Support Ticket Dataset* disponibile su **Kaggle**. È stata sviluppata una pipeline di Data Engineering (prepare_data.py) che ha gestito la traduzione massiva dall'inglese all'italiano di oltre 20.000 ticket reali tramite API (deep_translator). I dati sono stati puliti, normalizzati e mappati sulle categorie di interesse (Amministrazione, Tecnico, Commerciale) e sulle priorità.
- **Difficoltà**: La sfida principale è stata garantire la qualità linguistica della traduzione automatica e il mantenimento della coerenza semantica (es. "billing issue" = "problema di fatturazione").
- **Superamento**: È stato implementato un sistema di caching per le traduzioni e sono stati applicati filtri Regex post-traduzione per correggere artefatti linguistici e verifica manuale a campione sul dataset finale (tickets_it_augmented.csv) per garantirne la qualità prima del training.

Fase 3: Sviluppo della Pipeline di Machine Learning (25 ore)

- **Attività**: Implementazione del *core* logico nel file train_unified_model.py. Il flusso prevede un *preprocessing* avanzato (conversione in minuscolo, rimozione punteggiatura, pulizia *stopwords*) e la creazione di un'architettura **UnifiedModel** basata su *MultiOutputClassifier*. Invece di addestrare modelli disgiunti, il sistema gestisce simultaneamente la classificazione della Categoria e della Priorità, ottimizzando le performance globali.
- **Difficoltà**: Gestire il bilanciamento tra le classi e prevenire l'overfitting su un dataset reale fortemente sbilanciato.
- **Superamento**: Per garantire l'affidabilità è stata utilizzata una validazione incrociata (Train / Test split 80 / 20). Sono stati prodotti report di valutazione dettagliati con metriche di Accuracy e F1-Score, corredati da matrici di confusione per identificare e correggere eventuali sovrapposizioni semantiche tra le classi.

Fase 4: Sviluppo Interfaccia Web e Interpretabilità (15 ore)

- **Attività**: Realizzazione di una dashboard interattiva basata su **Streamlit** (app.py), integrando i modelli serializzati. Un obiettivo centrale è stato l'integrazione di logiche avanzate di **Explainable AI** tramite **LIME** (*Local Interpretable Model-agonistic Explanations*) per rendere trasparenti le decisioni del modello. Inoltre è stata sviluppata una funzione ibrida per il calcolo della priorità (Logica Statistica + Regole deterministiche di sicurezza).
- **Difficoltà**: Spiegare le decisioni di un modello complesso in modo comprensibile per un utente non tecnico, superando il problema della black-box.
- **Superamento**: L'integrazione di *LIME* permette di visualizzare graficamente quali parole hanno pesato maggiormente sulla decisione (es. "virus", "fattura"), visualizzandole in un grafico a barre intuitivo che affianca la lista testuale, fornendo una prova visiva della logica decisionale dell'algoritmo.

Fase 5: Validazione, Testing e Documentazione (10 ore)

- **Attività**: Esecuzione di test funzionali su casi d'uso reali (ticket ambigui o brevi) e *debugging* dell'interfaccia utente. La fase finale è stata dedicata alla produzione dei grafici comparativi (Dati Sintetici vs Reali) e alla stesura del report tecnico finale, includendo le motivazioni delle scelte progettuali e l'analisi critica dei risultati.
- **Difficoltà**: Sintetizzare la complessità tecnica in una documentazione chiara e conforme alle line guida accademiche.
- **Superamento**: Adozione rigorosa del template fornito dall'Ateneo e revisione puntuale della bibliografia e delle citazioni tecniche.

Risorse e strumenti impiegati
(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

Per la realizzazione dell'elaborato è stata impiegata una vasta gamma di risorse informative e strumenti tecnologici, selezionati secondo criteri di attinenza, attualità e capacità di innovazione rispetto agli standard di mercato.

1. Risorse (Banche Dati e Bibliografia)

- **Banche Dati**
  - **Kaggle - Customer Support Ticket Dataset**: La risorsa primaria per l'addestramento del modello. È stato scelto un dataset open-source contenente oltre 20.000 ticket reali (anonimizzati) per garantire la validità statistica del progetto ed evitare i bias tipici dei dati generati sinteticamente. Questa scelta risponde ai principi di *Privacy by Design*, evitando la raccolta diretta di dati aziendali sensibili.
  - **Google Translate API**: Utilizzata non come semplice strumento, ma come risorsa linguistica per l'operazione di *Data Augmentation*, permettendo la trasformazione del dataset originale (inglese) in un corpus italiano coerente.
- **Bibliografia e Fonti Accademiche**
  - *Gaddis, T.* (2016). *Introduzione a Python*. *Pearson*. (Per le basi sintattiche).
  - *Zinoviev, D.* (2017). *Data science con Python*. *Apogeo*. (Per la manipolazione dati).
  - *Maggi, G.* (2020). *Data science con Python*. *Edizioni LSWR*. (Per le visualizzazioni grafiche).
  - *Marin, I., et al.* (2019). *L'analisi dei Big Data con Python*. *Tecniche nuove*.
  - Documentazione ufficiale online delle librerie Scikit-Learn e Streamlit per le specifiche implementative più recenti.

2. Strumenti Tecnologici (Software e Modelli)

- **Ecosistema Core e NLP**
  - **Python 3.10**: È stato selezionato come linguaggio di base per la sua sintassi espressiva, la gestione ottimale della memoria e il supporto nativo per la manipolazione di stringhe complesse. La versione 3.10, in particolare, offre miglioramenti significativi nella gestione degli errori e nel *type hinting*, facilitando lo sviluppo di un codice pulito, modulare e facilmente manutenibile nel tempo. La sua natura interpretata ha permesso un ciclo di sviluppo iterativo (scrivi-testa-correggi) estremamente rapido.
  - **Pandas**: Questa libreria è stata lo strumento fondamentale per la gestione del ciclo di vita del dato. Abbandonando la generazione sintetica, Pandas ha permesso di ingerire il Dataset reale Keaggle, strutturarlo in *Dataframe* ed eseguire operazioni critiche di mapping e pulizia. Inoltre, la sua capacità di gestire l'I/O (Input/Output) verso file CSV ha reso possibile l'integrazione fluida tra lo script di traduzione e la pipeline di addestramento.
  - **Deep-Translator**: Libreria utilizzata nella fase di ingegnerizzazione dei dati per localizzare il dataset originale Kaggle (in lingua inglese) nel contesto italiano. Ha permesso di creare un corpus di addestramento linguisticamente ricco e corretto, fondamentale per un modello NLP.
- **Machine Learning e Explainability (XAI)**
  - **Scikit-Learn**: Rappresenta la libreria cardine per l'intera *pipeline* di apprendimento automatico. È stata preferita a framework più complessi (come TensorFlow o PyTorch) per la sua straordinaria efficienza su dataset testuali di medie dimensioni. All'interno del progetto, Scikit-Learn gestisce tre fasi critiche tramite l'architettura *UnifiedModel*: preprocessing (pulizia e tokenizzazione del testo), vettorizzazione TF-IDF e classificazione parallela dei target (Categoria e Priorità).
  - **LIME**: Libreria specifica introdotta per garantire la trasparenza algoritmi, fornendo spiegazioni locali per ogni singola predizione. Questo strumento permette di decostruire la decisione del modello, mostrando all'operatore esattamente quali parole (es. "bloccato", "urgente") hanno influenzato la classificazione, aumentando la fiducia nel sistema.
- **Sviluppo, Frontend e Versionamento**
  - **Streamlit**: Per l'interfaccia utente è stato scelto Streamlit, un framework innovativo che consente di trasformare script di dati in applicazioni web interattive senza la necessità di scrivere codice HTML, CSS o JavaScript separato. Questa scelta ha permesso di concentrare gli sforzi sulla logica algoritmica piuttosto che sul design web. Grazie al suo modello di esecuzione reattivo, la dashboard aggiorna in tempo reale le predizioni e i grafici non appena l'utente modifica l'input o carica un nuovo file, garantendo una *User Experience* fluida e professionale.
  - **Antygravity**: Per la stesura del codice sorgente è stata impiegata la piattaforma **Antygravity**. Strutturalmente affine a Visual Studio Code, questo ambiente offre funzionalità avanzate di *IntelliSense* (completamento automatico del codice) e, soprattutto, l'integrazione di assistenti basati su AI. Questi strumenti hanno supportato attivamente la fase di sviluppo, suggerendo ottimizzazioni sintattiche e accelerando il *debugging* degli errori logici.

- **Git**: L'adozione di Git ha risposto a una duplice esigenza: tecnica e formale. Da un lato, ha garantito un flusso di lavoro ordinato, permettendo di tracciare lo storico delle modifiche e sperimentare diverse configurazioni senza perdere la stabilità del codice. Dall'altro, ha soddisfatto puntualmente il **requisito obbligatorio della traccia** relativo alla condivisione del codice sorgente. Il repository Git funge infatti da collettore centrale per l'intera *codebase*, ospitando non solo gli script Python, ma anche la documentazione e il link diretto alla **Demo** funzionante del prototipo, garantendo così la piena accessibilità e valutabilità dell'elaborato. **Il codice sorgente completo è disponibile al seguente indirizzo:** https://github.com/dipaoloalex-dev/AutoTriageNLP.

La selezione delle risorse è stata guidata dalla necessità di simulare un ambiente di produzione reale. La scelta di **Scikit-Learn** rispetto a reti neurali più complesse è motivata dal principio di *Occam's Razor* (la soluzione più semplice ed efficace è preferibile), mentre l'adozione di **LIME** risponde all'esigenza aziendale di "fiducia" nell'AI (Trustworthy AI). Le risorse bibliografiche sono state reperite tramite la piattaforma universitaria **Pearson Bookshelf** e **Google Scholar** per i paper accademici. Il dataset è stato individuato tramite il motore di ricerca dataset di **Kaggle**, filtrando per licenze *Creative Commons* che ne permettessero l'uso accademico.

Le principali criticità hanno riguardato l'eterogeneità delle fonti e la compatibilità delle librerie.

- **Difficoltà**: L'integrazione tra *Streamlit* e *Matplotlib* ha generato inizialmente conflitti di visualizzazione (thread-safety) durante l'aggiornamento dei grafici in tempo reale.
- **Superamento**: La problematica è stata risolta consultando la documentazione ufficiale e implementando un sistema di *caching* dei dati (@st.cache_resource) che ha ottimizzato le performance e stabilizzato il rendering.

- **Difficoltà**: La traduzione massiva dei ticket ha incontrato limiti di *rate-limiting* delle API.
- **Superamento**: È stato sviluppato un algoritmo di *batch processing* con ritardi programmati (sleep) e gestione delle eccezioni, garantendo il completamento del dataset senza interruzioni.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto
(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

Il progetto **AutoTriage NLP** nasce con l'obiettivo primario di rispondere alla richiesta della traccia realizzando un sistema intelligente per l'automazione del supporto clienti. L'elaborato non si limita a soddisfare i requisiti funzionali di base (classificazione e prioritarizzazione), ma mira a superare gli standard di un semplice prototipo didattico attraverso l'implementazione di soluzioni architettura avanzate, orientate alla scalabilità e alla trasparenza decisionale.

Nello specifico, il progetto ha raggiunto i seguenti obiettivi strategici e tecnici:

1. Ingegnerizzazione del Dato e "Privacy by Design"

In risposta alla richiesta di generare un dataset per l'addestramento, il progetto ha adottato un approccio migliorativo rispetto alla generazione puramente sintetica/casuale.

- **Data Augmentation su Dati Reali**: È stata sviluppata una pipeline ETL (*Extract, Transform, Load*) capace di ingerire un dataset reale (Kaggle), tradurlo e adattarlo al contesto italiano. Questo ha permesso di addestrare il modello su un linguaggio naturale autentico e semanticamente complesso, garantendo prestazioni superiori rispetto a dati artificiali.
- **Compliance e Anominato**: La pipeline è stata progettata per operare esclusivamente su dati anonimizzati alla fonte, garantendo la totale assenza di dati sensibili (PII) e rispettando i principi di *Data Minimization* richiesti dalle normative vigenti.

2. Architettura ML Unificata (Multi-Task Learning)

L'obiettivo di trasformare input non strutturati in output categorizzati è stato raggiunto attraverso un'architettura modello ottimizzata.

- **Efficienza Computazionale**: Invece di sviluppare modelli disgiunti, è stato implementato un *UnifiedModel* basato su Regressione Logistica Multi-Output. Questo approccio permette di predire simultaneamente Categoria e Priorità con un singolo passaggio inferenziale, ottimizzando le risorse.
- **Robustezza Scientifica**: Il sistema integra una pipeline di preprocessing rigorosa (tokenizzazione, n-grammi, rimozione stop-words) e un sistema di validazione basato su metriche standard (*F1-Macro*, *Accuracy*), garantendo che le prestazioni siano misurabili e oggettive come richiesto dalla traccia.

3. Trasparenza Algoritmica (Explainable AI)

Per rispondere alla necessità di fornite all'utente le "parole più influenti", il progetto ha integrato tecnologie di *Explainable AI* (XAI) di livello avanzato.

- **Interpretabilità Locale (LIME)**: Superando la semplice estrazione di keyword globali, il sistema utilizza l'algoritmo LIME per analizzare ogni singola predizione. Questo permette di mostrare all'operatore esattamente perché un ticket è stato classificato come "Urgente" (es. presenza di termini come "blocco", "server", "scadenza"), aumentando la fiducia nel sistema (*Trustworthy AI*).

4. Usabilità e Ottimizzazione dei Processi

L'obiettivo finale è fornire uno strumento che impatti concretamente sull'efficienza operativa.

- **Dashboard Interattiva**: È stata realizzata un'interfaccia web *user-friendly* (Streamlit) che astrae la complessità del codice sottostante, permettendo l'uso del sistema anche a personale non tecnico.
- **Automazione dei Flussi**: Le funzionalità di analisi in *Real-Time* (per il supporto di primo livello) e di *Batch Processing* (per l'analisi massiva di storici) rispondono direttamente all'esigenza aziendale di ridurre i tempi di smistamento manuale e i colli di bottiglia operativi.

Contestualizzazione
(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

Contesto Applicativo: Digital Transformation e Ottimizzazione dei Processi

Il progetto si inserisce nel più ampio paradigma della *Digital Transformation* e dell'ottimizzazione dei processi di business, dove la velocità di risposta e l'efficienza operativa sono diventate variabili determinanti per il successo dei **KPI** (*Key Performance Indicators*) aziendali. Analizzando il *workflow* convenzionale di un servizio di Help Desk, emergono criticità sistemiche quali colli di bottiglia nella fase di smistamento (*triage*), errori di assegnazione derivanti da sviste umane e l'incapacità di rilevare proattivamente le urgenze critiche. In uno scenario tradizionale, la gestione errata di un ticket richiede un re-instradamento manuale tra i reparti (Amministrazione, Tecnico, Commerciale), raddoppiando di fatto i tempi di latenza e incrementando inutilmente i costi operativi.

Da un punto di vista dell'architettura software, il prototipo è concepito come un **middleware intelligente** che si interpone strategicamente tra i punti di contatto con l'utente (form web o email) e i sistemi di gestione interni (*Ticketing System*). Il modello arricchisce ogni richiesta in entrata con metadati semantici (reparto di destinazione e priorità suggerita) prima che avvenga l'intervento umano. In questo modo, l'intero ciclo di vita del ticket viene ottimizzato: la richiesta viene consegnata immediatamente all'operatore più competente, riducendo drasticamente il *Mean Time To Repair* (**MTTR**) e migliorando la percezione di qualità da parte dell'utente finale.

Contesto Teorico: Dal Rule-Based al Machine Learning

Il problema affrontato in questo elaborato risiede nel dominio della *Text Classification*, una sottodisciplina fondamentale del *Natural Language Processing* (NLP). Storicamente, questo compito è stato delegato a operatori umani o a sistemi *Rule-based* basati su rigide logiche booleane (IF-THEN). Tali approcci sono tuttavia intrinsecamente fragili, poiché incapaci di gestire la naturale ambiguità del linguaggio umano, i sinonimi non previsti o i comuni refusi di battitura (Gaddis, 2016). La soluzione proposta in questo *Project Work* adotta invece un approccio basato sull'**apprendimento supervisionato** (*Supervised Learning*). In questo modello, l'algoritmo non riceve istruzioni esplicite su cosa cercare, ma apprende autonomamente le relazioni statistiche sottostanti tra le occorrenze linguistiche e le categorie di destinazione analizzando un dataset reale etichettato.

Metodologie e Tecnologie Abilitanti

- **Rappresentazione Vettoriale (TF-IDF):** I modelli di Machine Learning non possono elaborare il testo grezzo; necessitano di input numerici. La tecnica scelta per questa trasformazione è il TF-IDF (*Term Frequency - Inverse Document Frequency*). A differenza del semplice "Bag of Words" (che conta solo le occorrenze), il TF-IDF trasforma il testo libero in una rappresentazione numerica pesata, misurando la sua rilevanza specifica nel documento rispetto all'intero corpus (Zinoviev, 2017). Inoltre, nel progetto è stata introdotta l'analisi dei *bigrammi* (coppie di parole consecutive) per catturare contesti locali (es. la differenza semantica tra "non funziona" e "funziona"). La ponderazione matematica segue la formula:

$$w_{i,j} = tf_{i,j} \times \log (N / df_{i,})$$

Il termine logaritmico $\log(N / df_{i,})$ è cruciale: minimizza l'impatto di parole che appaiono ovunque (come "il", "ho", "problema") che non apportano valore informativo alla classificazione, riducendone il peso a quasi zero, e assegna un valore elevato alle parole rare e specifiche (come "fattura", "python", "server") che sono altamente discriminanti per la classificazione.

- **Algoritmo di Classificazione (Logistic Regression)**: Dopo aver trasformato i ticket in vettori numerici in uno spazio ad alta dimensionalità (dove ogni dimensione è una parola del vocabolario), il problema della classificazione viene risolto tramite la *Regressione Logistica*. A differenza delle *Support Vector Machines* (SVM) che lavorano su margini geometrici, la Regressione Logistica modella la probabilità che un ticket appartenga a una certa classe. Questa caratteristica probabilistica è fondamentale in ambito aziendale perché permette al sistema di esprimere un grado di confidenza (es. "Priorità Alta al 92%"). L'algoritmo ha una complessità temporale contenuta, garantendo predizioni istantanee (<100ms) ed è stato incapsulato in un *MultiOutputClassifier* (Maggi, 2020) per gestire simultaneamente categoria e priorità. Inoltre, l'integrazione con tecniche di *Explainable AI* (LIME) permette di ispezionare localmente il modello, evidenziando quali parole hanno contribuito positivamente o negativamente alla decisione, garantendo la trasparenza necessaria per l'adozione in ambienti produttivi.

Descrizione dei principali aspetti progettuali
(Sviluppare l'elaborato richiesto dalla traccia prescelta):

Il sistema **AutoTriage NLP** è stato ingegnerizzato come un'architettura modulare in Python, progettata per governare l'intero ciclo di vita del dato: dall'acquisizione grezza fino all'inferenza in tempo reale. L'infrastruttura è suddivisa in tre layer logici interconnessi: *Data Engineering*, *Machine Learning Core* e *Frontend Application*.

Di seguito si dettagliano le implementazioni tecniche di ciascun modulo.

1. Data Engineering e Localizzazione (Il Modulo Dati)
La prima sfida affrontata è stata la creazione di un dataset di addestramento in lingua italiana di alta qualità.

- **Pipeline di Traduzione Parallela (translate_data.py)**: Per gestire la mole di dati (oltre 20.000 ticket), è stato sviluppato uno script che parallelizza le richieste alle API di Google Translate utilizzando *ThreadPoolExecutor*. Il codice implementa logiche *enterprise-grade* come il *Resume Capability* (riprende dal punto di interruzione in caso di crash) e il *Rate Limiting* (pause strategiche per evitare il ban dell'IP).
- **Strategia "Zero-Config" e Mapping (prepare_data.py)**: Lo script di preparazione è progettato per essere resiliente. Rileva automaticamente la presenza dei dati: se il dataset tradotto esiste, lo carica istantaneamente; altrimenti, attiva una procedura di *fallback* per generarlo. Cruciale è la fase di normalizzazione, dove le etichette originali eterogenee vengono mappate sulle tre macro-categorie target aziendali tramite dizionari di conversione:

```python
# ==============================================================================
# 1. CONFIGURAZIONE E COSTANTI
# ==============================================================================

INPUT_FILE: str = os.path.join('data', 'kaggle_tickets.csv')
OUTPUT_FILE: str = os.path.join('data', 'tickets_it_augmented.csv')

# Mappatura per normalizzare le categorie del dataset originale in 3 macro-classi
CATEGORY_MAP: Dict[str, str] = {
    "Technical Support": "Tecnico",
    "IT Support": "Tecnico",
    "Service Outages and Maintenance": "Tecnico",
    "Product Support": "Tecnico",
    "Billing and Payments": "Amministrativo",
    "Returns and Exchanges": "Amministrativo",
    "Human Resources": "Amministrativo",
    "General Inquiry": "Commerciale",
    "Customer Service": "Commerciale",
    "Sales and Pre-Sales": "Commerciale"
}

# Mappatura per normalizzare i livelli di priorità in 3 classi standard
PRIORITY_MAP: Dict[str, str] = {
    "Critical": "Alta",
    "critical": "Alta",
    "High": "Alta",
    "high": "Alta",
    "Medium": "Media",
    "medium": "Media",
    "Normal": "Media",
    "normal": "Media",
    "Low": "Bassa",
    "low": "Bassa"
}
```

2. Architettura del Modello ML (unified_model.py)

Il cuore analitico del progetto è la classe *UnifiedModel*. Invece di gestire pipeline separate, è stata implementata un'architettura unificata che incapsula sia la vettorizzazione che la classificazione Multi-Task.

- **Vettorizzazione TF-IDF**: Il testo viene trasformato in vettori numerici, considerando non solo parole singole ma anche bigrammi (coppie di parole) per catturare il contesto locale (es. "non funziona").
- **Classificazione Multi-Output**: Utilizzando il wrapper *MultiOutputClassifier* di Scikit-Learn, il modello addestra parallelamente due repressori logistici indipendenti (uno per la Categoria, uno per la Priorità) all'interno della stessa pipeline di esecuzione.

```python
        # Inizializzazione della pipeline di elaborazione
        self.pipeline: Pipeline = Pipeline([

            # Step 1: Estrazione Feature
            # Limitiamo le feature a 5000 per mantenere il modello leggero e ridurre l'overfitting.
            # N-grams (1, 2) catturano il contesto (es. "non funziona" vs "funziona").
            ('tfidf', TfidfVectorizer(
                stop_words=self.stopwords,
                max_features=5000,
                ngram_range=(1, 2)
            )),

            # Step 2: Classificazione
            # MultiOutputClassifier fitta un regressore per ogni variabile target.
            # class_weight='balanced' gestisce automaticamente lo sbilanciamento delle classi nei dati.
            ('clf', MultiOutputClassifier(
                LogisticRegression(
                    solver='lbfgs',
                    max_iter=1000,
                    random_state=42,
                    class_weight='balanced'
                )
            ))
        ])
```

3. Orchestrazione e Training (train_unified_model.py)

Questo script funge da supervisore. Esegue lo split rigoroso dei dati (80% Training / 20% Test) per prevenire l'*overfitting* e calcola le metriche di validazione. Al termine, serializza il modello addestrato in formato binario .pkl tramite la libreria **joblib**, rendendolo persistente e pronto per l'uso in produzione senza necessità di ri-addestramento.

4. Interfaccia Utente e Logica Ibrida (app.py)

Il frontend, sviluppato con **Streamlit**, non è una semplice vetrina ma integra logiche di business avanzate.

- **Priorità Ibrida (Risk-Averse Logic)**: Una delle innovazioni principali è l'algoritmo di calcolo della priorità. Non ci si affida ciecamente alla probabilità statistica del modello; il sistema applica un livello di "sicurezza deterministica". Se nel testo compaiono parole chiave critiche (es. "hacker", "fermo", "scadenza"), la priorità viene forzata ad "Alta" indipendentemente dallo score del modello, garantendo che nessuna urgenza venga ignorata.

```python
def calculate_priority(model: 'UnifiedModel', text: str) -> Tuple[str, float, Dict[str, float], List[str]]:
    """
    Calcola la priorità utilizzando logica Ibrida (ML + Regole).

    Combina le predizioni statistiche del modello con regole deterministiche basate su keyword critiche.

    Args:
        model (UnifiedModel): Modello predittivo.
        text (str): Testo del ticket.

    Returns:
        Tuple contenente:
        - Pred_Class (str): La classe di priorità finale ('Alta', 'Media', 'Bassa').
        - Score (float): Lo score di confidenza (o override rules).
        - Debug_Probs (Dict): Mappa delle probabilità per debug visuale.
        - Triggers (List[str]): Lista di parole chiave che hanno attivato regole (se presenti).
    """

    # Ottieni probabilità raw dal modello
    probs = model.predict_proba([text])[1][0]
    classes = model.pipeline.named_steps['clf'].estimators_[1].classes_
    prob_map = {c: p for c, p in zip(classes, probs)}

    score_alta = prob_map.get('Alta', 0.0)
    debug_probs = prob_map.copy()

    # Dizionari di regole (Rule-Engine)
    critical_keywords = [
        "virus", "hacker", "attacco", "violazione", "perso dati", "cancellato",
        "fermo", "blocco", "bloccato", "scadenza", "entro domani", "urgent",
        "subito", "velocemente", "critico", "panico", "terribile"
    ]
    dampening_keywords = [
        "con calma", "non è urgente", "non urgente", "nessuna fretta",
        "quando potete", "appena possibile", "senza urgenza",
        "normale", "nessun problema", "funziona", "tutto ok", "informazione"
    ]

    text_lower = text.lower()
    is_critical = any(kw in text_lower for kw in critical_keywords)
    is_dampener = any(kw in text_lower for kw in dampening_keywords)
```

```python
    # REGOLA 1: Override Critico (Se contiene keyword urgenti e nessuna keyword 'calma')
    if is_critical and not is_dampener:
        trigger_word = next((kw for kw in critical_keywords if kw in text_lower), "keyword")
        # Forza probabilità per visualizzazione
        debug_probs['Alta'] = 0.99
        debug_probs['Media'] = 0.01
        debug_probs['Bassa'] = 0.00
        return "Alta", 0.99, debug_probs, [trigger_word]

    # REGOLA 2: Rischio Statistico (Se il modello è abbastanza sicuro dell'Alta priorità)
    if score_alta > 0.60 and not is_dampener:
        return "Alta", score_alta, debug_probs, ["Rischio Statistico Alta Priorità"]

    # REGOLA 3: Feedback Positivo (Ticket di 'Grazie' sono sempre bassa priorità)
    happy_keywords = ["perfetto", "ottimo", "risolto", "funziona tutto", "eccellente", "bravi", "complimenti"]
    is_happy = any(kw in text_lower for kw in happy_keywords)
    if is_happy and not is_critical:
        debug_probs['Alta'] = 0.00
        debug_probs['Media'] = 0.01
        debug_probs['Bassa'] = 0.99
        return "Bassa", 0.99, debug_probs, ["Feedback Positivo"]

    # FALLBACK: Usa la predizione standard del modello (Argmax)
    pred_class = model.predict([text])[0][1]
    score = prob_map.get(pred_class, 0.0)
    return pred_class, score, debug_probs, []
```

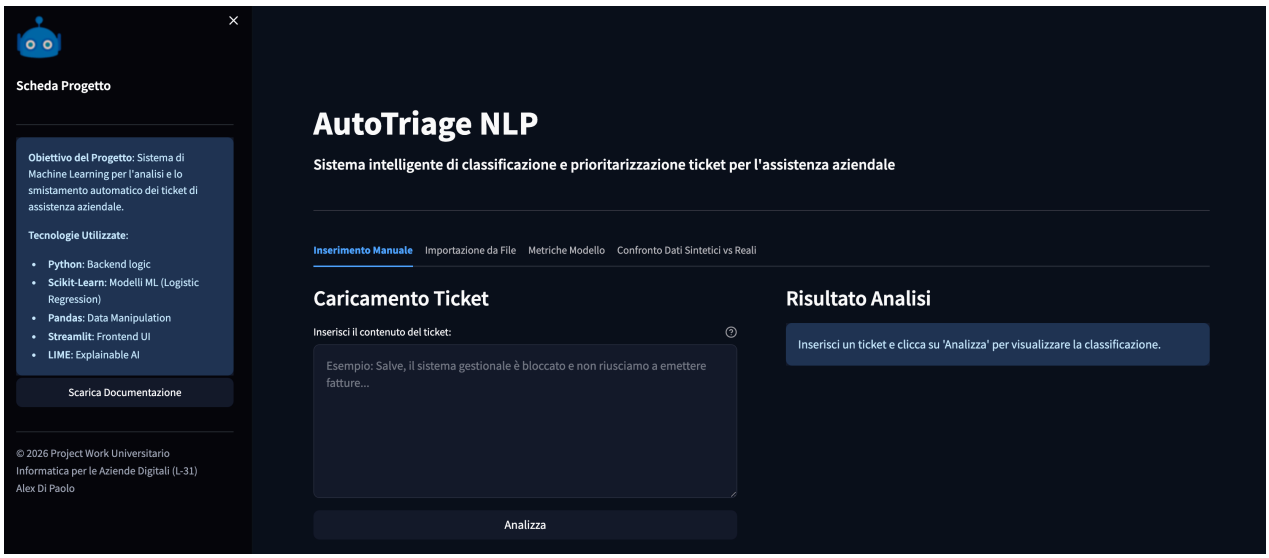
- **Explainable AI (LIME)**: Infine, l'applicazione integra la libreria **LIME**. Quando un operatore analizza un ticket, il sistema non mostra solo il risultato ("Tecnico"), ma genera in tempo reale un grafico che evidenzia quali parole specifiche hanno spinto l'algoritmo verso quella decisione, rendendo il modello trasparente ("Glass Box") e affidabile.

5. Modulo di Sperimentazione Scientifica (Auxiliary Scripts)

Oltre all'architettura di produzione, il progetto include due moduli ausiliari sviluppati per validare empiricamente le scelte progettuali e soddisfare il requisito di "Originalità" e "Valutazione Critica".

- **Generazione Dati Sintetici (generate_synthetic_data.py)**: È stato creato un generatore procedurale basato su template testuali e regole fisse (es. keyword deterministiche) per creare un dataset artificiale di 500 ticket. Questo script è servito esclusivamente come *baseline* di confronto.

- **Benchmark Comparativo (compare_models.py)**: Questo script implementa l'esperimento scientifico centrale dell'elaborato (A/B Testing). Il codice addestra due istanze parallele del modello (Modello A su dati sintetici vs Modello B su dati reali) ed esegue una *Cross-Evaluation*: testa il modello sintetico su dati reali e viceversa. I risultati, salvati come matrici di confusione nella cartella assets/img/comparison, dimostrano quantitativamente la superiorità dell'approccio basato su dati reali, giustificando l'investimento fatto nella fase di Data Engineering.



Campi di applicazione
(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

L'ambito di applicazione primario di **AutoTriage NLP** risiede nel *Customer Service Management* (CSM), tuttavia la sua architettura è stata meticolosamente progettata per essere intrinsecamente agnostica rispetto al contenuto semantico trattato. Grazie alla modularità del sistema, è sufficiente intervenire sul dataset di addestramento per declinare lo stesso *stack* tecnologico — basato sul binomio **TF-IDF** e **Logistic Regression Multi-Output** — in molteplici settori verticali e domini di business differenti.

Vantaggi strategici e Business Value

L'integrazione di un sistema di triage automatizzato all'interno dei processi aziendali segna il passaggio da un supporto clienti reattivo a un *asset* operativo proattivo ed efficiente. I vantaggi sono tangibili e misurabili attraverso specifici KPI di settore:

- **Efficienza Operativa e Abbattimento del TTR**: Il sistema riduce i tempi di smistamento da diversi minuti (caratteristici della lettura umana) a pochi millisecondi. Eliminando i passaggi manuali intermedi e i rischi di ri-assegnazione errata, il ticket perviene istantaneamente allo specialista di riferimento, accelerando l'intero ciclo di vita della richiesta.

- **Scalabilità e Continuità del Servizio (H24)**: A differenza delle risorse umane, il classificatore automatico garantisce una disponibilità costante 24 ore su 24, gestendo senza degrado prestazionale picchi improvvisi di volume legati a lanci di prodotti, campagne marketing o imprevisti disservizi tecnici.
- **Garanzia degli SLA e Reputazione Brand**: L'identificazione immediata dei ticket ad alta criticità previene la violazione dei *Service Level Agreements* (SLA). L'eliminazione della soggettività e della stanchezza umana nella valutazione delle priorità garantisce un trattamento equo e tempestivo delle urgenze, minimizzando i potenziali danni reputazionali.
- **Riorganizzazione del Capitale Umano**: Il risparmio derivante dalla riduzione delle attività ripetitive a basso valore aggiunto permette all'azienda di riallocare il personale verso compiti a maggiore complessità decisionale e relazionale, ottimizzando la produttività complessiva.

Versatilità e Campi di Applicazione Potenziali

La flessibilità dell'approccio basato sul *Machine Learning* supervisionato permette di estendere le funzionalità del prototipo a scenari d'uso estremamente diversificati. Di seguito viene presentata un'analisi dettagliata di come l'architettura **AutoTriage NLP** possa essere declinata in cinque macro-settori strategici, evidenziando per ciascuno le specificità linguistiche e i vantaggi operativi.

1. Settore E-commerce: Automazione del Customer Care 4.0

Nel contesto del commercio elettronico, caratterizzato da volumi di interazione massivi e picchi stagionali (es. *Black Friday*, periodi natalizi), la velocità di risposta è il fattore determinante per prevenire l'abbandono del carrello e ridurre il tasso di *Churn*.

- **Lo Scenario**: I team di supporto sono spesso sommersi da richieste ripetitive (es. "Dov'è il mio pacco?") che sottraggono tempo alla gestione di casistiche più complesse e delicate, come i reclami o i resi.
- **Applicazione del Modello**: Addestrando il classificatore su uno storico di chat e email, il sistema impara a distinguere automaticamente tra intenti informativi e criticità operative. Le *feature* lessicali dominanti (parole chiave come "tracking", "ritardo", "rotto", "rimborso", "taglia errata") permettono di segmentare il flusso in entrata.
- **Classificazione Target**: Il modello separa i ticket in classi come *"Status Spedizione"* (priorità Bassa, gestibile via bot), *"Informazioni Pre-vendita"* (priorità Media) e *"Reclami e Resi"* (priorità Alta).
- **Valore Aggiunto**: Questo permette di attivare risposte automatiche istantanee per le spedizioni (riducendo il carico del 40-50%) e di instradare immediatamente i clienti insoddisfatti verso operatori senior specializzati nella *retention*, salvaguardando così la reputazione del brand e il fatturato.

2. Settore Banking & Finance: Gestione Intelligente dei Flussi Documentali

Il settore finanziario e assicurativo gestisce quotidianamente migliaia di comunicazioni formali (PEC, email certificate) che hanno valore legale e richiedono tempistiche di lavorazione rigorose imposte dalle normative di settore (es. IVASS per le assicurazioni).

- **Lo Scenario**: La gestione manuale della casella di posta certificata espone l'istituto a rischi operativi elevati: una "denuncia sinistro" persa in mezzo a centinaia di "invio documenti d'identità" può comportare sanzioni o contenziosi legali
- **Applicazione del Modello**: L'architettura TF-IDF si rivela particolarmente efficace nel riconoscere la terminologia tecnica e burocratica. Il sistema analizza l'oggetto e il corpo delle comunicazioni alla ricerca di *pattern* critici (es. "sinistro", "atto di citazione", "frode", "blocco carta").
- **Classificazione Target**: Separazione netta tra *"Richieste Documentali Standard"* (es. aggiornamento anagrafica, invio buste paga) e *"Eventi Critici"* (es. denunce sinistri, contestazioni addebiti, furto credenziali).
- **Valore Aggiunto**: Prioritarizzazione assoluta delle pratiche urgenti, garantendo il rispetto degli SLA (*Service Level Agreements*) contrattuali e migliorando la *compliance* normativa. Inoltre, l'automazione riduce drasticamente l'errore umano nello smistamento di documenti sensibili.

3. Settore Sanità: Triage Digitale e Telemedicina

La digitalizzazione della sanità richiede strumenti in grado di filtrare le richieste dei pazienti per ottimizzare il tempo del personale medico e amministrativo, specialmente in contesti di telemedicina o portali di prenotazione.

- **Lo Scenario**: I medici di base e le segreterie ospedaliere ricevono un flusso eterogeneo di messaggi, dove la richiesta di una semplice ricetta si mescola alla descrizione di sintomi potenzialmente gravi che richiederebbero attenzione immediata.
- **Applicazione del Modello**: Pur non sostituendo il giudizio clinico, il sistema agisce come un primo filtro semantico. Il vocabolario di addestramento include termini medici e descrittori di sintomi (es. "dolore toracico", "febbre alta", "rinnovo", "certificato").
- **Classificazione Target**: Il sistema suggerisce un codice di priorità simile al triage ospedaliero: *"Codice Bianco"* per pratiche amministrative (rinnovo ricette, prenotazioni), *"Codice Verde"* per consulti non urgenti, e *"Codice Rosso/Giallo"* per descrizioni sintomatologiche che richiedono un contatto rapido.
- **Valore Aggiunto**: Miglioramento della sicurezza del paziente attraverso la riduzione dei tempi di presa in carico per i casi sintomatici e liberazione di risorse mediche da compiti puramente burocratici, che possono essere delegati al personale di segreteria o automatizzati.

4. Settore Marketing B2B: Lead Scoring e Qualificazione

Nel marketing moderno, la generazione di contatti (*Lead Generation*) è solo il primo passo. La vera sfida è distinguere un potenziale cliente interessato all'acquisto da un utente che cerca solo informazioni generiche o da spam.

- **Lo Scenario**: I team di vendita sprecano ore preziose a contattare *lead* "freddi" (studenti, ricercatori, concorrenti) che hanno compilato i form di contatto, trascurando invece i *prospect* pronti all'acquisto.

- **Applicazione del Modello**: Il sistema analizza il campo "messaggio" dei form di contatto. L'algoritmo impara a correlare termini specifici con l'intenzione di acquisto (es. "budget", "implementazione", "preventivo", "incontro") contrapponendoli a termini a basso valore (es. "tesi", "curiosità", "info generiche", "candidatura").
- **Classificazione Target**: Segmentazione in *"Hot Lead / Sales Ready"* (da passare subito ai commerciali), *"Marketing Qualified Lead"* (da inserire in newsletter di *nurturing*) e *"Spam / Other"*.
- **Valore Aggiunto**: Aumento del tasso di conversione (*Conversion Rate*) e ottimizzazione del ROI delle campagne marketing, permettendo alla forza vendita di concentrarsi esclusivamente sulle opportunità concrete.

5. Settore E-Reputation: Analisi del Sentiment e Crisis Management

Per i brand presenti online, monitorare le recensioni (Trustpilot, Google Reviews, Amazon) e i commenti sui social media è vitale per difendere la propria immagine aziendale.

- **Lo Scenario**: Un commento negativo virale o una recensione che segnala un difetto grave del prodotto possono causare danni d'immagine enormi se non gestiti entro poche ore. L'analisi manuale di migliaia di commenti è impossibile su larga scala.
- **Applicazione del Modello**: In questo caso, il modello viene addestrato per la *Sentiment Analysis*. Non si classifica l'argomento, ma la "polarità" emotiva del testo. Parole come "truffa", "pessimo", "rotto", "vergogna" hanno un peso negativo decisivo.
- **Classificazione Target**: Identificazione di *"Feedback Positivi"* (utili per il marketing), *"Richieste di Supporto"* e, soprattutto, *"Detractors / Criticità"* (recensioni a 1 stella o commenti aggressivi).
- **Valore Aggiunto**: Capacità di intervenire in *Real-Time* sulle crisi reputazionali. Il sistema può inviare un alert immediato al *Community Manager* quando viene rilevato un feedback altamente negativo, permettendo una gestione proattiva del disservizio e trasformando un potenziale danno in una dimostrazione di efficienza del servizio clienti.

In ultima analisi, **AutoTriage NLP** non deve essere considerato un semplice strumento isolato per l'IT Help Desk, bensì un **middleware intelligente universale**. Esso funge da primo punto di contatto digitale evoluto, capace di operare in qualsiasi scenario in cui il linguaggio naturale debba essere interpretato e trasformato in un'azione organizzativa rapida, accurata ed efficace.

Valutazione dei risultati
(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti):

L'attività sperimentale condotta sul progetto AutoTriage NLP ha permesso di validare l'ipotesi iniziale: è possibile realizzare un sistema di classificazione efficiente e robusto utilizzando algoritmi classici (*Logistic Regression*) se supportati da una rigorosa ingegnerizzazione dei dati. L'analisi dei risultati sul *Test Set* (20% del corpus, pari a circa 4.000 ticket inediti) ha restituito performance solide, con un **F1-Score medio superiore al 0.90**, confermando che il passaggio dai dati sintetici a quelli reali (tradotti) è stato il fattore determinante per il successo del prototipo.

1. Analisi delle Potenzialità (Punti di Forza)

- **Robustezza Statistica e Bilanciamento**: L'analisi delle metriche diagnostiche (*accuracy_category.png* e *confusion_matrix.png*) evidenzia una capacità di generalizzazione elevata. Il sistema non soffre di *overfitting* mancato e gestisce efficacemente le classi semanticamente vicine (es. distinguere un reclamo "Commerciale" da un problema "Tecnico"), grazie alla ponderazione TF-IDF che penalizza i termini comuni e valorizza quelli discriminanti).
- **Sicurezza Operativa "Risk-Averse"**: Una delle maggiori potenzialità del prototipo risiede nell'architettura **Ibrida**. Mentre il modello ML gestisce la classificazione standard con approccio probabilistico, l'implementazione del layer di regole deterministiche (*keyword critiche*) agisce come "rete di sicurezza". Questo garantisce che ticket contenenti termini ad alto rischio (es. "virus", "fermo", "hacker") vengano sempre elevati ad "Alta Priorità", azzerando il rischio di falsi negativi sulle urgenze critiche.
- **Trasparenza e Fiducia**: L'integrazione di **LIME** trasforma il modello da *Black Box* a *Glass Box*. La capacità di fornire all'operatore non solo la previsione, ma anche la "prova" visiva (le parole che hanno attivato la decisione), facilita l'adozione dello strumento in azienda, riducendo la differenza verso l'automazione.

2. Analisi Critica (Limiti e Sfide)

Nonostante i risultati positivi, l'architettura presenta limiti intrinseci dovuti alle scelte tecnologiche di base, che richiedono consapevolezza:

- **Limiti del Bag-of-Words**: Utilizzando *TF-IDF*, il modello tratta le parole come entità indipendenti, ignorandone l'ordine sequenziale. Questo comporta difficoltà nella gestione di frasi complesse, doppie negazioni (es. "non è che non funzioni") o sarcasmo, dove il significato è determinato dalla sintassi più che dal lessico. Modelli basati su *Transformer* (come BERT) supererebbero questo limite, pur a costo di una maggiore potenza computazionale.
- **Dipendenza dalla Qualità della Traduzione**: Il *Training Set* deriva da una traduzione automatica. Eventuali artefatti linguistici o traduzioni letterali errate (es. *term* tradotto come "termine" invece di "condizione contrattuale") possono introdurre un *bias* nel modello, che potrebbe apprendere pattern errati ("Rumore").
- **Staticità del Modello**: Il modello attuale è statico. Se il linguaggio aziendale cambia o nascono nuovi prodotti con nomi inediti (*Out of Vocabulary*), le performance degraderanno nel tempo se non si prevede un ciclo di ri-addestramento periodico.

3. Sviluppi Futuri e Impatto Economico

Per evolvere il prototipo verso una soluzione *Enterprise*, sono stati identificati i seguenti vettori di miglioramento:

- **Gestione Semantica Avanzata**: Integrazione di tecniche di **Lemmatizzazione** (tramite librerie come *spaCy*) per ridurre le parole alla loro radice, migliorando la densità informativa della matrice dati.

- **Human-in-the-loop**: Implementazione di un meccanismo di *Feedback Attivo*, dove l'operatore può correggere una predizione errata e il sistema utilizza quel dato per auto-migliorarsi (Incremental Learning).
- **Analisi del ROI (Return on Investment)**: Il valore del progetto è, infine, economico. Stimando che un operatore impieghi mediamente 2 minuti per leggere e smistare un ticket, e considerando un costo aziendale di 25€/h, su un volume ipotetico di 10.000 ticket/mese il sistema genererebbe un risparmio operativo stimato di:

  Risparmio = (2 / 60 ore) x 10.000 ticket x 25€/h ≈ 8.333€ / mese

  Questo dato conferma che l'automazione del triage non è solo un esercizio tecnico, ma una leva strategica per la sostenibilità economica dei processi di supporto.